DVA494

Programming of Reliable Embedded Systems

Obed Mogaka | Hamid Mousavi | Masoud Daneshtalab

IDD, Malardalens University

March 9, 2026

Today's Agenda

- OnChip Memories on the FPGA
  - Distributed Memory (MLUT-based)
  - Block RAM-based (BRAM-based)
- Inferring RAM and ROM in VHDL

Memory Technology on FPGAs

- FF:
  - Single flip-flop, 1 bit of memory
- LUT RAM:
  - LUT6: 64 bit RAM
  - 4LUTs per CLB: 256 bit RAM
  - SRL32: 32 bit shift memory
- RAM blocks:
  - AMD: BlockRAM 36 kbit
    UltraRAM 288 kbits
  - Altera: M9K, M10K, M144K, ..
- External memory
  - SSRAM: QDR SRAM
  - SDRAM: DDR3, DDR4, ..

Memory Features

Feature	FF	LUT RAM	BlockRAM	UltraRAM
Init value	✓	✓	✓	✓
Reset	✓	×	×	×
Dual port	✓	✓	✓	✓
Cross clock	✓	✓	✓	×
Sync. write	✓	✓	✓	✓
Sync. read	async	async	✓	✓

NOTE:

- Avoid building memory from single FF (device utilization)
- Do not reset RAMs, because it doesn't exist ⇒ otherwise single FF are used
- UltraRAMs can't be used in cross clock FIFOs
- Sync. write + sync. read requires 2 clock cycles

Distributed vs Block RAMs

FPGAs offer two distinct types of on-chip memory, each optimized for different tasks.

1. Distributed RAM

- **Implementation:** Constructed using the **Look-Up Tables (LUTs)** inside the Logic Blocks.
- **Capacity:** Very small (bits to kilobits).
- **Behavior:** Can be asynchronous (combinational read) or synchronous.
- **Best For:** Small scratchpad registers, shallow FIFOs, and coefficient storage close to the logic.

2. Block RAM (BRAM)

- **Implementation:** Dedicated hard silicon memory blocks embedded in columns across the device.
- **Capacity:** Large (typically 18Kb or 36Kb per block).
- **Behavior:** Strictly **Synchronous** (requires a clock).
- **Best For:** Large data buffers, image line storage, and main memory for soft-core processors.

Key Trade-off: Use Distributed RAM for speed and locality; use Block RAM for density.

AMD Artix-7 FPGA Resources

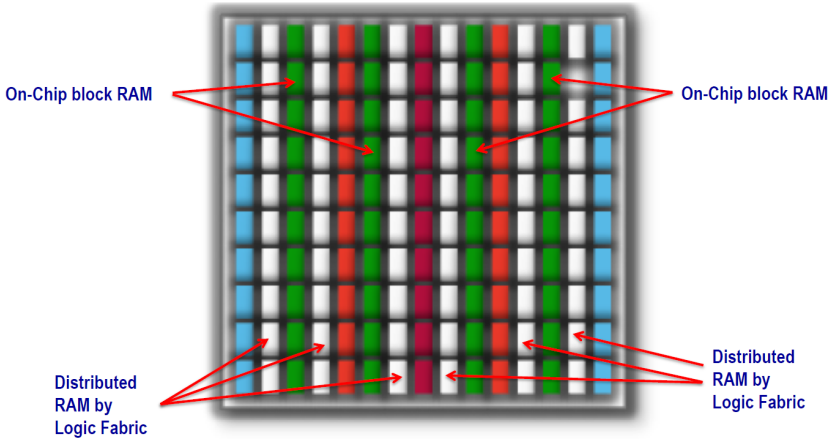
Table 1: **XA Artix-7 FPGA Device-Feature Table**

Device	Logic Cells	Configurable Logic Blocks (CLBs)		DSP48E1 Slices[2]	Block RAM Blocks[3]			CMTs[4]	PCIe[5]	GTPs	XADC Blocks	Total I/O Banks[6]	Max User I/O[7]
		Slices[1]	Max Distributed RAM (Kb)		18 Kb	36 Kb	Max (Kb)						
XA7A12T	12,800	2,000	171	40	40	20	720	3	1	2	1	3	150
XA7A15T	16,640	2,600	200	45	50	25	900	5	1	4	1	5	210
XA7A25T	23,360	3,650	313	80	90	45	1,620	3	1	4	1	3	150
XA7A35T	33,280	5,200	400	90	100	50	1,800	5	1	4	1	5	210
XA7A50T	52,160	8,150	600	120	150	75	2,700	5	1	4	1	5	210
XA7A75T	75,520	11,800	892	180	210	105	3,780	6	1	4	1	6	285
XA7A100T	101,440	15,850	1,188	240	270	135	4,860	6	1	4	1	6	285

Notes:

1. Each 7 series FPGA slice contains four LUTs and eight flip-flops; only some slices can use their LUTs as distributed RAM or SRLs.
2. Each DSP slice contains a pre-adder, a 25 x 18 multiplier, an adder, and an accumulator.
3. Block RAMs are fundamentally 36 Kb in size; each block can also be used as two independent 18 Kb blocks.
4. Each CMT contains one MMCM and one PLL.
5. XA Artix-7 FPGA Interface Blocks for PCI Express support up to x4 Gen 2.
6. Does not include configuration Bank 0.
7. This number does not include GTP transceivers.

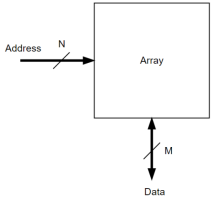
FPGA Memory Resources

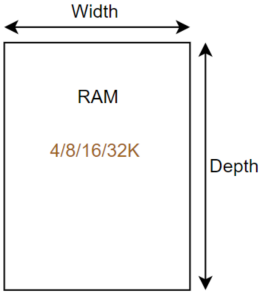


RAM Features

- Memory is an area of bits per word $\times$ number of words
- A memory array is described in terms of depth and width:
  - Depth: No. of rows (No. of words)
  - Width: No. of columns (Size of word)

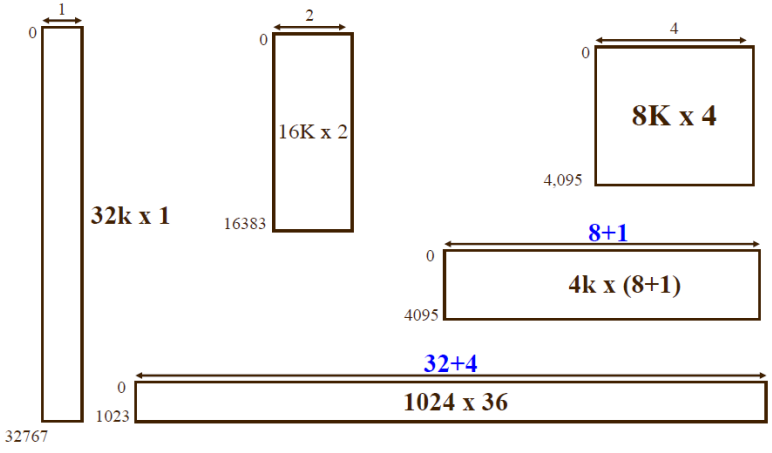
$$ArraySize = depth \times width = 2^N \times M$$
$$N = Address\ bits$$
$$M = Data\ bits$$





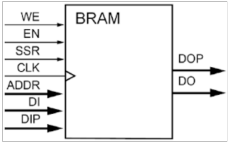
Block RAM Configurations

Aspect Ratios:



BRAM Configurations: Port Modes

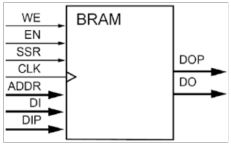
1. Single Port (SP)



- One port for both Read and Write.
- Sequential access only.
- *Use case:* Basic storage.

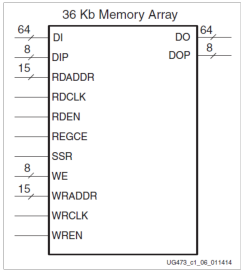
BRAM Configurations: Port Modes

1. Single Port (SP)



- One port for both Read and Write.
- Sequential access only.
- *Use case:* Basic storage.

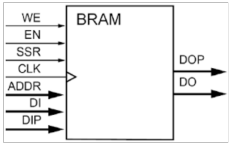
2. Simple Dual Port (SDP)



- **Port A:** Write Only.
- **Port B:** Read Only.
- Simultaneous R/W allowed.
- *Use case:* FIFOs.

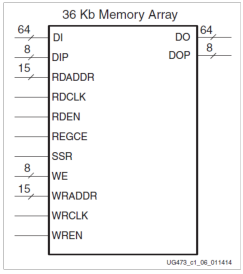
BRAM Configurations: Port Modes

1. Single Port (SP)



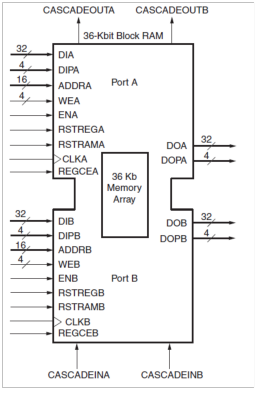
- One port for both Read and Write.
- Sequential access only.
- *Use case:* Basic storage.

2. Simple Dual Port (SDP)



- **Port A:** Write Only.
- **Port B:** Read Only.
- Simultaneous R/W allowed.
- *Use case:* FIFOs.

3. True Dual Port (TDP)



- Two fully independent R/W ports.
- Can access any address from any port.
- *Use case:* Shared memory.

Handling RAMs: Inference vs. Instantiation

There are two methods to implement RAM in your FPGA design: writing generic HDL (**Inference**) or manually placing specific hardware blocks (**Instantiation**).

1. Inference (Recommended)

"Write standard VHDL arrays and let the synthesizer map them to RAM."

Advantages:

- **Portable Code:** The same code works on Xilinx, Intel, or Lattice devices.
- **Easier Workflow:** No external tools or IP generation steps required.
- **Timing-Driven:** The synthesis tool automatically optimizes the memory layout for your clock constraints.

2. Instantiation

"Directly call a vendor-specific RAM primitive or IP Core."

Advantages:

- **Maximum Efficiency:** You get exactly the hardware usage you expect.
- **Advanced Features:** Access to specific hardware capabilities (like built-in ECC or specialized FIFO logic) that inference might miss.
- **Total Control:** Supports all kinds of RAM configurations.

VHDL Coding for Memory

VHDL Coding: Defining Memory Arrays

In VHDL, we do not have a built-in "memory" primitive. Instead, we model memory by creating a **2-Dimensional Array** of standard logic vectors.

The 2-Step Process:

1. **Define a new Type:** Create an array where each index represents an address, and each element is a data word.
2. **Declare a Signal/Constant:** Create a physical instance of that type to hold your data.

VHDL Array Syntax

```vhdl
-- Example: 16 words, 8 bits wide
type memory_array is array (0 to 15)
    of std_logic_vector(7 downto 0);

-- For RAM: Declare as a signal
signal my_ram : memory_array;

-- For ROM: Declare as a constant
constant my_rom : memory_array := (
    0 => x"AA",
    1 => x"BB",
    -- ...
    others => x"00"
);
```

Important: We use `ieee.numeric_std.all` so we can convert the binary `address` input into an integer to index the array.

Distributed ROM with Asynchronous Read

- The data output updates immediately when the address changes (combinational logic).
- **Hardware Map:** The synthesizer will map this directly to **Distributed RAM (LUTs)** configured as ROM. It cannot be mapped to BRAM because BRAM requires a clock.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL; -- Needed for to_integer and unsigned

entity async_rom is
    Port (
        address  : in  STD_LOGIC_VECTOR(1 downto 0); -- 4 words = 2 addr bits
        data_out : out STD_LOGIC_VECTOR(7 downto 0)  -- 8-bit word size
    );
end async_rom;

architecture Behavioral of async_rom is
    -- 1. Define the ROM type
    type rom_type is array (0 to 3) of STD_LOGIC_VECTOR(7 downto 0);

    -- 2. Define the constant and initialize it
    constant MY_ROM : rom_type := (
        0 => "00001111",
        1 => "11110000",
        2 => "10101010",
        3 => "01010101"
    );
begin
    -- 3. Asynchronous Read (No Process, No Clock)
    data_out <= MY_ROM(to_integer(unsigned(address)));

end Behavioral;
```

Implementing Distributed RAM (LUT RAM)

Distributed RAM is constructed by repurposing the Look-Up Tables (LUTs) within the FPGA's logic slices (specifically SLICEM in AMD/Xilinx architectures)

Key Architectural Traits:

- **Write Operation: Strictly Synchronous.** Data is written on the active clock edge.
- **Read Operation: Asynchronous (Combinational).** Output data appears immediately after the address changes, independent of the clock.

Expert Design Tip:

- Use for shallow memory depths (e.g., 64 words or less).
- Excellent for low-latency scratchpad memory because the read operation does not cost a clock cycle.

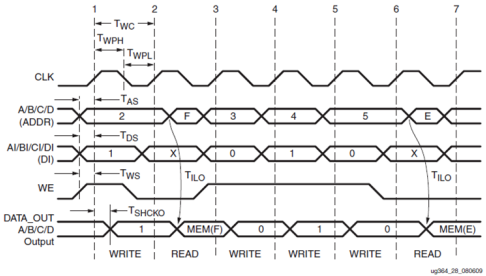


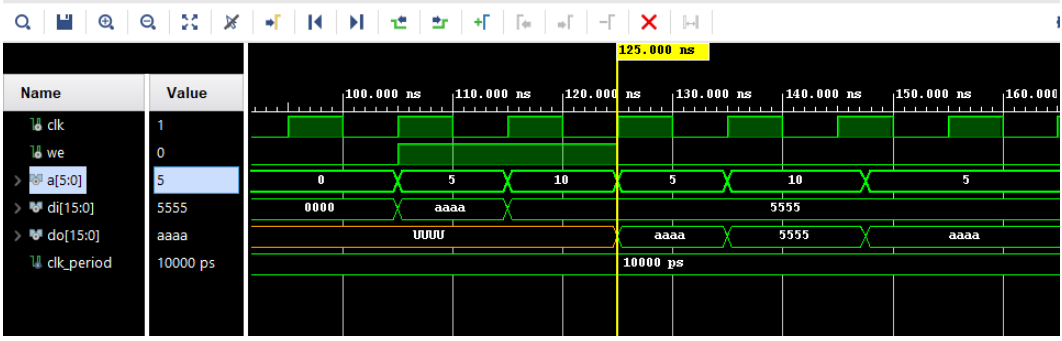
Figure 28: **Slice Distributed RAM Timing Characteristics**

VHDL Inference: Single-Port Distributed RAM

The Golden Rule of Inference: To force the synthesizer to map your array to Distributed RAM, place the Read assignment **outside** the clocked process.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity dist_ram_sp is
    Port ( clk  : in  STD_LOGIC;
           we   : in  STD_LOGIC; -- Write Enable
           addr : in  STD_LOGIC_VECTOR(5 downto 0); -- 64 words
           di   : in  STD_LOGIC_VECTOR(15 downto 0);
           do   : out STD_LOGIC_VECTOR(15 downto 0));
end dist_ram_sp;

architecture RTL of dist_ram_sp is
    type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(15 downto 0);
    signal RAM : ram_type;
begin
    -- 1. Synchronous Write (Inside Process)
    process(clk)
    begin
        if rising_edge(clk) then
            if we = '1' then
                RAM(to_integer(unsigned(addr))) <= di;
            end if;
        end if;
    end process;

    -- 2. Asynchronous Read (Outside Process)
    -- This specific line dictates LUT RAM mapping!
    do <= RAM(to_integer(unsigned(addr)));
end RTL;
```

Simulation: Distributed RAM: Sync Write, Async Read



On-Chip Block RAM (BRAM): Core Concepts

Block RAMs are dedicated, hard-silicon memory blocks embedded in the FPGA fabric. Unlike Distributed RAM, BRAM operations (both Read and Write) are **strictly synchronous**.

The Write Collision Problem:

- What happens to the `data_out` port when you Read and Write to the *exact same address* in the *same clock cycle*?
- FPGAs handle this via configurable **Operating Modes**.

On-Chip Block RAM (BRAM): Core Concepts

Block RAMs are dedicated, hard-silicon memory blocks embedded in the FPGA fabric. Unlike Distributed RAM, BRAM operations (both Read and Write) are **strictly synchronous**.

The Write Collision Problem:

- What happens to the `data_out` port when you Read and Write to the *exact same address* in the *same clock cycle*?
- FPGAs handle this via configurable **Operating Modes**.

BRAM Operating Modes:

1. **Write-First (Read-Through):** The new data being written is immediately passed to the output port.
2. **Read-First (Read-Before-Write):** The old data currently in the memory is placed on the output port before it is overwritten.
3. **No-Change:** The output port remains unchanged during a write operation (saves power).

Note

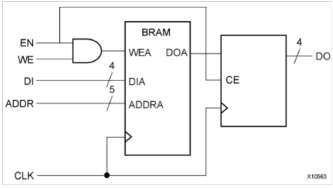
The operating mode is not set by a GUI checkbox when inferring memory; it is dictated entirely by *how you arrange your VHDL signal assignments* inside the clocked process.

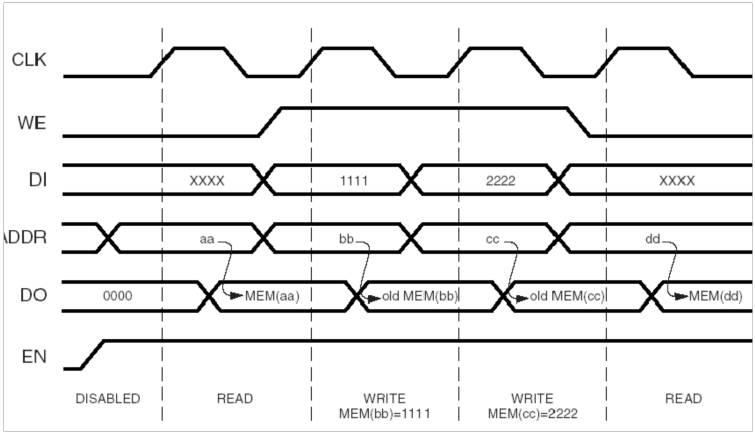
VHDL Inference: Single-Port BRAM

To force BRAM, **both** the memory write and the memory read must be inside the `rising_edge(clk)` condition.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ocram_sp is
    generic (
        A_BITS : integer := 10; -- 1024 words
        D_BITS : integer := 32  -- 32-bit words
    );
    port (
        clk  : in  std_logic;
        ena  : in  std_logic; -- RAM Enable
        we   : in  std_logic; -- Write Enable
        addr : in  std_logic_vector(A_BITS-1 downto 0);
        din  : in  std_logic_vector(D_BITS-1 downto 0);
        dout : out std_logic_vector(D_BITS-1 downto 0)
    );
end ocram_sp;

architecture RTL of ocram_sp is
    type ram_type is array (0 to (2**A_BITS)-1) of std_logic_vector(D_BITS-1 downto 0);
    signal RAM : ram_type;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if ena = '1' then
                if we = '1' then
                    RAM(to_integer(unsigned(addr))) <= din;
                end if;
                -- Read inside the clock process forces BRAM!
                -- This specific arrangement infers "Read-First" mode.
                dout <= RAM(to_integer(unsigned(addr)));
            end if;
        end if;
    end process;
end RTL;
```

Block RAM with Sync. Read (Read-First Mode)

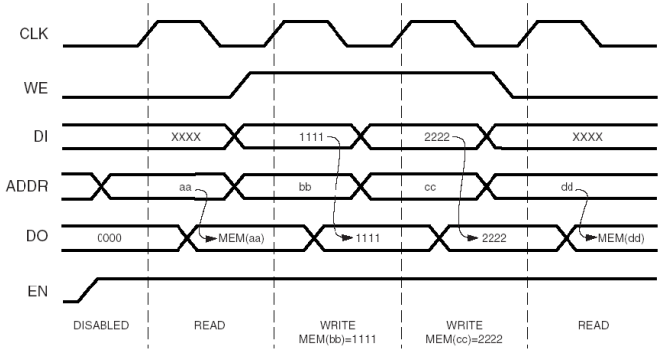




Block RAM: Write-First Mode (Read-Through)

In **Write-First** mode, the data being written to the memory array is simultaneously passed directly to the output port `do`.

```
process (clk)
begin
  if rising_edge(clk) then
    if (en = '1') then
      if (we = '1') then
        -- 1. Write to memory
        RAM(to_integer(unsigned(addr))) <= di;
        -- 2. Pass input directly to output
        do <= di;
      else
        -- Normal read
        do <= RAM(to_integer(unsigned(addr)));
      end if;
    end if;
  end if;
end process;
```

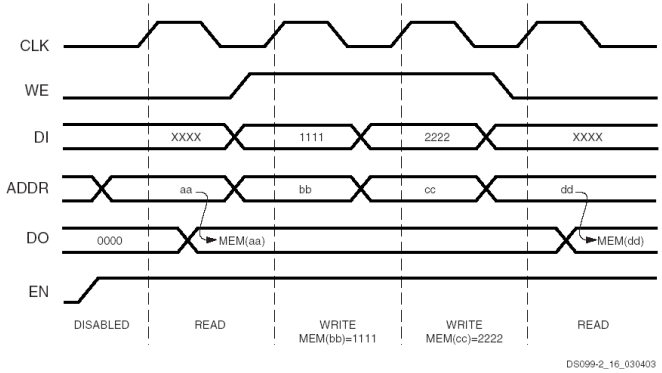


Notice how `do` updates immediately with `di` when `we = '1'`.

Block RAM: No-Change Mode

In **No-Change** mode, the output port `do` retains its previous value during a write operation. This is often the **most power-efficient mode** because the output bus does not toggle unnecessarily.

```
process (clk)
begin
  if rising_edge(clk) then
    if (en = '1') then
      if (we = '1') then
        -- 1. Write to memory ONLY
        RAM(to_integer(unsigned(addr))) <= di;
        -- Notice: 'do' is NOT updated here!
      else
        -- Normal read
        do <= RAM(to_integer(unsigned(addr)));
      end if;
    end if;
  end if;
end process;
```



Notice how `do` ignores `di` and holds its last read value when `we = '1'`.

Initialization of Block RAM

While BRAMs can start empty (or zeroed), we often need to preload them with data (e.g., Bootloader code, filter coefficients, or sine wave lookup tables).

Initialization Methods:

1. **Hardcoded Constants:** Good for very small ROMs, but pollutes HDL code for large arrays.
2. **File I/O (Recommended):** Using VHDL `std.textio` to read a `.mif` or `.hex` file during synthesis. The synthesis tool embeds this data directly into the bitstream.

Initializing Block RAM from an External File

How it Works:

- We use an **impure function**. It is "impure" because reading a file is a side-effect outside the function's scope.
- **Synthesis-Only:** This function executes *only* during Vivado synthesis. The tool reads the file and embeds the 1s and 0s directly into the FPGA bitstream.

The Data File (`ram_init.data`):

- Must be added to your Vivado project sources.
- One binary (or hex) word per line.

Example File Content

```
0000111100001111
1010101010101010
... (256 lines total)
```

VHDL Implementation:

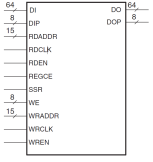
```vhdl
library STD;
use STD.textio.all;

architecture RTL of ocram_sp is
  type ram_type is array (0 to 255) of std_logic_vector(15 downto 0);

  -- Function reads file line-by-line
  impure function initramfromfile(ramfilename : in string) return ram_type
        is
    file ramfile       : text open read_mode is ramfilename;
    variable ramline   : line;
    variable ram_mem   : ram_type;
    variable bitvec    : bit_vector(15 downto 0);
  begin
    for i in ram_type'range loop
      readline(ramfile, ramline);
      read(ramline, bitvec); -- Parse binary string
      ram_mem(i) := to_stdlogicvector(bitvec);
    end loop;
    return ram_mem;
  end function;

  -- 1. Initialize the RAM signal by calling the function
  signal RAM : ram_type := initramfromfile("ram_init.data");
begin
  -- 2. Standard BRAM Process follows here...
```

Block RAM: Simple Dual Port (SDP) Inference

A Simple Dual Port (SDP) RAM dedicates **Port A exclusively for writing** and **Port B exclusively for reading**. This allows simultaneous read and write operations to different addresses.

Key Architectural Traits:

- **Independent Clocks:** Port A and Port B can be driven by completely asynchronous clocks (`clka` and `clkb`).
- **Use Case:** This is the exact hardware structure used to build **Asynchronous FIFOs** for safely passing data between different clock domains.
- **Collision Warning:** If `clka` and `clkb` are asynchronous, reading and writing to the *exact same address* simultaneously yields undefined read data.



VHDL Inference (Dual-Clock SDP):

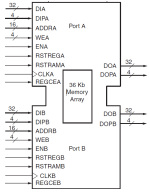
```vhdl
architecture RTL of ocram_sdp is
  type ram_type is array (0 to 1023) of
       std_logic_vector(31 downto 0);
  signal RAM : ram_type;
  -- Note: RAM is driven by ONLY ONE process (Port A)
begin
  -- PORT A: Write Only Process
  process(clka)
  begin
    if rising_edge(clka) then
      if ena = '1' and wea = '1' then
        RAM(to_integer(unsigned(addra))) <= dina;
      end if;
    end if;
  end process;

  -- PORT B: Read Only Process
  process(clkb)
  begin
    if rising_edge(clkb) then
      if enb = '1' then
        -- Synchronous Read
        doutb <= RAM(to_integer(unsigned(addrb)));
      end if;
    end if;
  end process;
end RTL;
```

Block RAM: True Dual Port (TDP) Inference

A True Dual Port (TDP) RAM provides the maximum level of flexibility. Ports A and B are fully independent.

Key Architectural Traits:

- **Complete Independence:** Each port has its own address, data in/out buses, clock, and write enable (WE) signals.
- **Synchronous Access:** Both read and write operations on either port are strictly synchronous to their respective clocks.



Design Warning: Write Collisions

If both ports attempt to write to the *exact same address* simultaneously, it causes data uncertainty. The memory contents at that address become undefined. Resolution logic must be handled externally in your design!

VHDL Inference (TDP):

```vhdl
architecture RTL of ocram_tdp is
  type ram_type is array (0 to 1023) of
       std_logic_vector(31 downto 0);
  shared variable RAM : ram_type;
  -- Note: Some synthesis tools prefer 'signal' here,
  -- but 'shared variable' formally resolves
  -- multi-process driving in VHDL-93.
begin
  -- PORT A: Read/Write Process
  process(clka)
  begin
    if rising_edge(clka) then
      if ena = '1' then
        if wea = '1' then
          RAM(to_integer(unsigned(addra))) := dina;
        end if;
        douta <= RAM(to_integer(unsigned(addra)));
      end if;
    end if;
  end process;

  -- PORT B: Read/Write Process
  process(clkb)
  begin
    if rising_edge(clkb) then
      if enb = '1' then
        if web = '1' then
          RAM(to_integer(unsigned(addrb))) := dinb;
        end if;
        doutb <= RAM(to_integer(unsigned(addrb)));
      end if;
    end if;
  end process;
end RTL;
```